Task 1 Report

(It was mentioned in the doubt doc that)

Task 1.2: Code Quality Analysis (Pylint)

The objective of this task was to improve the codebase quality based on Pylint feedback. A perfect score of 10.00/10 was achieved after 10 iterations.

Iteration 1:

- Removed MACOSX folder
- Removed extra space from cards.py

Iteration 2:

- Few extra space removed from cards.py
- Added all missing docstrings

Iteration 3:

- Fixed all unnecessary import and unnecessary keyword/paren errors
- Removed all unused variables

Iteration 4:

- Changed == to is for singleton comparison in board.py to check if a property is mortgaged

Iteration 5:

- Fixed all final new line missing errors

Iteration 6:

- Grouped all the jail related instances into one in player.py
- Removed mortgage_vale and houses instances from property.py. Calculating mortgage when needed and houses is not needed there.
- Added the chance and community decks to board.py from game.py

Iteration 7:

- Removed f-string when not needed.

Iteration 8:

- Combined the “birthday” and “collect_from_all” card actions within _apply_card(). Since both cards performed the exact same logic, they could be grouped together with an in statement.
- Removed a redundant “if prop:”" check since title==“property” already ensures that prop is valid

Iteration 9:

- Initialized doubles_streak = 0 inside init in dice.py
- Specified the ValueError type in try-except in ui.py

Iteration 10:

- In property.py, init now accepts costs grouping price and base rent
- Updated the initialization list in board.py to pass these two identical values grouped in a tuple

---

Task 1.3: Whitebox Testing

Created 93 test cases (unit testing). A total of 21 logical bugs and runtime errors were progressively found and fixed. Below are the tests that initially failed, why they were required, and the specific fixes implemented to solve the underlying code flaws.

1. **`test_player_initialization`**

Why it is needed: Validates that the Player initialization functionality operates securely without boundary violations.

Result: `Passed`

2. **`test_player_dd_deduct_money`**

Why it is needed: Validates that the Player add deduct money functionality operates securely without boundary violations.

Result: `Passed`

3. **`test_player_move_passes_go`**

Why it is needed: Validates that the Player move passes go functionality operates securely without boundary violations.

Result: `Passed (Fixed)`

Bug/Error: AssertionError: Player should receive GO_SALARY when passing Go

Fix: Changed condition from `self.position == 0` to `old_position + steps >= BOARD_SIZE` so Go salary is awarded when passing Go, not just landing on it

4. **`test_player_net_worth`**

Why it is needed: Validates that the Player net worth functionality operates securely without boundary violations.

Result: `Passed (Fixed)`

Bug/Error: AssertionError: Net worth should include property values

Fix: Changed `return self.balance` to `return self.balance + sum(p.price for p in self.properties)` to include property values in net worth

5. **`test_player_go_to_jail`**

Why it is needed: Verifies accurate deduction and turn-handling when jailed.

Result: `Passed`

6. **`test_dice_roll_values`**

Why it is needed: Validates that the Dice roll values functionality operates securely without boundary violations.

Result: `Passed`

7. **`test_dice_max_roll`**

Why it is needed: Validates that the Dice max roll functionality operates securely without boundary violations.

Result: `Passed (Fixed)`

Bug/Error: AssertionError: Dice should be able to roll a 6 (faces 1-6)

Fix: Changed `random.randint(1, 5)` to `random.randint(1, 6)` for both dice to allow rolling a 6

8. **`test_dice_doubles`**

Why it is needed: Validates that the Dice doubles functionality operates securely without boundary violations.

Result: `Passed`

9. **`test_property_group_all_owned_by`**

Why it is needed: Validates that the Property group all owned by functionality operates securely without boundary violations.

Result: `Passed (Fixed)`

Bug/Error: AssertionError: Should only be true if ALL properties are owned by player

Fix: Changed `any()` to `all()` in `all_owned_by` so group ownership requires owning every property, not just one

10. **`test_property_mortgage`**

Why it is needed: Checks mortgage multipliers and nested active boolean state transitions.

Result: `Passed (Fixed)`

Bug/Error: assert mortgage_value == 100

Fix: Changed `return self.mortgage_value` to `return self.mortgage_value()` in `mortgage()` to call the method instead of returning the method object

11. **`test_property_rent`**

Why it is needed: Validates that the Property rent functionality operates securely without boundary violations.

Result: `Passed (Fixed)`

Bug/Error: AssertionError: assert 20 == 10 AND Error 17: Rent was doubling for owning a full group even if some members of that group were mortgaged (illegal in Monopoly rules).

Fix: Changed `any()` to `all()` in group ownership check, and added an extra validation layer to `get_rent` that verifies no members of the color group are mortgaged before applying the bonus.

12. **`test_bank_transactions`**

Why it is needed: Validates that the Bank transactions functionality operates securely without boundary violations.

Result: `Passed`

13. **`test_bank_loans`**

Why it is needed: Validates that the Bank loans functionality operates securely without boundary violations.

Result: `Passed (Fixed)`

Bug/Error: Assertion failure testing exact bank funds decrease, since Bank.give_loan failed to deduct the given loan amount from the bank's funds.

Fix: Added `self.pay_out(amount)` before `player.add_money()` in `give_loan` to ensure the bank's reserves correctly decrease by the loan amount constraint.

14. **`test_game_find_winner`**

Why it is needed: Validates that the Game find winner functionality operates securely without boundary violations.

Result: `Passed (Fixed)`

Bug/Error: AssertionError: find_winner should return the player with the highest net worth

Fix: Changed `min()` to `max()` in `find_winner` so the player with the highest net worth is returned

15. **`test_game_buy_property_exact_balance`**

Why it is needed: Validates that the Game buy property exact balance functionality operates securely without boundary violations.

Result: `Passed (Fixed)`

Bug/Error: AssertionError: Player should be able to buy property with exact balance

Fix: Changed `player.balance <= prop.price` to `player.balance < prop.price` so a player can buy when balance equals price exactly

16. **`test_game_pay_rent_transfers_money`**

Why it is needed: Validates that the Game pay rent transfers money functionality operates securely without boundary violations.

Result: `Passed (Fixed)`

Bug/Error: AssertionError: Owner should receive the rent payment

Fix: Added `prop.owner.add_money(rent)` in `pay_rent` so the owner actually receives the rent amount

17. **`test_game_jail_fine`**

Why it is needed: Verifies accurate deduction and turn-handling when jailed.

Result: `Passed (Fixed)`

Bug/Error: AttributeError: 'Player' object has no attribute 'jail_turns'

Fix: Changed `player.jail_turns` to `player.jail["turns"]` to access jail turns from the jail dict

18. **`test_game_bankruptcy`**

Why it is needed: Ensures bankruptcy processing correctly handles nested player elimination paths and properties are released gracefully.

Result: `Passed`

19. **`test_game_apply_card`**

Why it is needed: Validates that the Game apply card functionality operates securely without boundary violations.

Result: `Passed (Fixed)`

Bug/Error: AttributeError: 'Player' object has no attribute 'get_out_of_jail_cards'

Fix: Changed `player.get_out_of_jail_cards += 1` to `player.jail["cards"] += 1` to use the jail dict

20. **`test_get_player_names`**

Why it is needed: Validates that the Get player names functionality operates securely without boundary violations.

Result: `Passed`

21. **`test_main_success`**

Why it is needed: Validates that the Main success functionality operates securely without boundary violations.

Result: `Passed`

22. **`test_main_value_error`**

Why it is needed: Validates that the Main value error functionality operates securely without boundary violations.

Result: `Passed`

23. **`test_main_keyboard_interrupt`**

Why it is needed: Validates that the Main keyboard interrupt functionality operates securely without boundary violations.

Result: `Passed`

24. **`test_bank_methods`**

Why it is needed: Validates that the Bank methods functionality operates securely without boundary violations.

Result: `Passed`

25. **`test_bank_insufficient_funds`**

Why it is needed: Validates that the Bank insufficient funds functionality operates securely without boundary violations.

Result: `Passed`

26. **`test_board_repr`**

Why it is needed: Validates that the Board repr functionality operates securely without boundary violations.

Result: `Passed`

27. **`test_board_tile_types`**

Why it is needed: Validates that the Board tile types functionality operates securely without boundary violations.

Result: `Passed`

28. **`test_board_is_purchasable`**

Why it is needed: Validates that the Board is purchasable functionality operates securely without boundary violations.

Result: `Passed`

29. **`test_board_owned_and_unowned`**

Why it is needed: Validates that the Board owned and unowned functionality operates securely without boundary violations.

Result: `Passed`

30. **`test_card_deck_empty`**

Why it is needed: Validates that the Card deck empty functionality operates securely without boundary violations.

Result: `Passed`

31. **`test_card_deck_repr`**

Why it is needed: Validates that the Card deck repr functionality operates securely without boundary violations.

Result: `Passed (Fixed)`

Bug/Error: ZeroDivisionError: integer division or modulo by zero when `CardDeck` runs `__repr__` with an empty deck.

Fix: Added an explicit return statement `if not self.cards: return "CardDeck(0 cards, next=0)"` into `CardDeck.__repr__` to handle the empty case without hitting modulo zero exception.

32. **`test_player_remove_property`**

Why it is needed: Validates that the Player remove property functionality operates securely without boundary violations.

Result: `Passed`

33. `test_player_count_properties`

Why it is needed: Validates that the Player count properties functionality operates securely without boundary violations.

Result: `Passed`

34. `test_player_repr_and_status`

Why it is needed: Validates that the Player repr and status functionality operates securely without boundary violations.

Result: `Passed`

35. `test_player_bankrupt`

Why it is needed: Ensures bankruptcy processing correctly handles nested player elimination paths and properties are released gracefully.

Result: `Passed`

36. `test_property_repr`

Why it is needed: Validates that the Property repr functionality operates securely without boundary violations.

Result: `Passed`

37. `test_property_group_repr`

Why it is needed: Validates that the Property group repr functionality operates securely without boundary violations.

Result: `Passed`

38. `test_property_group_get_owner_counts`

Why it is needed: Validates that the Property group get owner counts functionality operates securely without boundary violations.

Result: `Passed`

39. `test_property_unmortgage_error`

Why it is needed: Checks mortgage multipliers and nested active boolean state transitions.

Result: `Passed`

40. `test_property_all_owned_by_none`

Why it is needed: Validates that the Property all owned by none functionality operates securely without boundary violations.

Result: `Passed`

41. `test_dice_repr`

Why it is needed: Validates that the Dice repr functionality operates securely without boundary violations.

Result: `Passed`

42. `test_ui_print_components`

Why it is needed: Intercepts deterministic raw standard inputs to secure game loop from malformed types.

Result: `Passed`

43. `test_ui_safe_int_input`

Why it is needed: Intercepts deterministic raw standard inputs to secure game loop from malformed types.

Result: `Passed`

44. `test_ui_safe_int_input_default`

Why it is needed: Intercepts deterministic raw standard inputs to secure game loop from malformed types.

Result: `Passed`

45. `test_ui_confirm`

Why it is needed: Intercepts deterministic raw standard inputs to secure game loop from malformed types.

Result: `Passed`

46. `test_game_handle_property_tile_buy`

Why it is needed: Validates that the Game handle property tile buy functionality operates securely without boundary violations.

Result: `Passed`

47. **`test_game_handle_property_tile_mortgaged`**

Why it is needed: Checks mortgage multipliers and nested active boolean state transitions.

Result: `Passed`

48. **`test_game_mortgage_menu`**

Why it is needed: Checks mortgage multipliers and nested active boolean state transitions.

Result: `Passed (Fixed)`

Bug/Error: TypeError: '<' not supported between instances of 'method' and 'int' AND Error 14: Bank.collect(-payout) was used to pay out mortgage funds from the bank, which failed after Bank.collect was fixed to ignore negative amounts.

Fix: Changed `self.mortgage_value` to `self.mortgage_value()` to call the method, and updated `mortgage_property` to use `bank.pay_out(payout)` for the payment instead of `collect(-payout)`.

49. **`test_game_unmortgage_menu`**

Why it is needed: Checks mortgage multipliers and nested active boolean state transitions.

Result: `Passed (Fixed)`

Bug/Error: TypeError: unsupported operand type(s) for *: 'method' and 'float'

Fix: Changed `prop.mortgage_value * 1.1` to `prop.mortgage_value() * 1.1` in `_menu_unmortgage` to call the method.

50. **`test_game_tax_tiles`**

Why it is needed: Validates that the Game tax tiles functionality operates securely without boundary violations.

Result: `Passed`

51. **`test_game_special_tiles`**

Why it is needed: Validates that the Game special tiles functionality operates securely without boundary violations.

Result: `Passed`

52. **`test_game_trade_menu_decline`**

Why it is needed: Ensures property exchanges correctly validate ownership and fund transfers.

Result: `Passed`

53. **`test_game_play_turn_basic`**

Why it is needed: Validates that the Game play turn basic functionality operates securely without boundary violations.

Result: `Passed (Fixed)`

Bug/Error: AttributeError: 'Player' object has no attribute 'in_jail' AND Error 19: `play_turn` used recursive calls for doubles which led to multiple turn advancements AND Error 20: No bankruptcy check at turn start led to negative balance moves.

Fix: Changed `in_jail` to dict access, replaced recursion with a `while True` loop to handle multiple rolls per turn safely, and added an initial balance check at turn start to eliminate bankrupt players.

54. **`test_game_jail_no_action`**

Why it is needed: Verifies accurate deduction and turn-handling when jailed.

Result: `Passed (Fixed)`

Bug/Error: AttributeError: 'Player' object has no attribute 'jail_turns' AND Error 21: Jail logic was missing the critical Monopoly rule that allows a player to try for doubles to escape jail for free.

Fix: Changed `jail_turns` to dict access, and implemented the `dice.roll()` attempt for doubles in `_handle_jail_turn` to ensure core rule compliance.

55. **`test_game_apply_card_other`**

Why it is needed: Validates that the Game apply card other functionality operates securely without boundary violations.

Result: `Passed`

56. **`test_game_interactive_limit`**

Why it is needed: Validates that the Game interactive limit functionality operates securely without boundary violations.

Result: `Passed`

57. **`test_game_run_loop_winner`**

Why it is needed: Validates that the Game run loop winner functionality operates securely without boundary violations.

Result: `Passed`

58. **`test_game_current_player`**

Why it is needed: Validates that the Game current player functionality operates securely without boundary violations.

Result: `Passed`

59. **`test_game_auction`**

Why it is needed: Validates that the Game auction functionality operates securely without boundary violations.

Result: `Passed (Fixed)`

Bug/Error: Error 18: Auction was implemented as a single fixed-round loop over players, which meant players had no opportunity to outbid someone who came after them in the list.

Fix: Refactored the auction logic into a proper continuous loop (`while True`) that only terminates once every other player has passed on the current highest bid.

60. **`test_mortgage_fail`**

Why it is needed: Checks mortgage multipliers and nested active boolean state transitions.

Result: `Passed`

61. **`test_trade_fail`**

Why it is needed: Ensures property exchanges correctly validate ownership and fund transfers.

Result: `Passed`

62. **`test_bank_edge_cases`**

Why it is needed: Validates that the Bank edge cases functionality operates securely without boundary violations.

Result: `Passed (Fixed)`

Bug/Error: Bank balance unexpectedly dropping when `Bank.collect` is supplied with a negative amount instead of silently ignoring it.

Fix: Added a preventative condition `if amount < 0: return` to the beginning of the `collect` method to respect the docstring bounds.

63. **`test_property_add_existing`**

Why it is needed: Validates that the Property add existing functionality operates securely without boundary violations.

Result: `Passed`

64. `test_ui_player_card_full`

Why it is needed: Intercepts deterministic raw standard inputs to secure game loop from malformed types.

Result: `Passed`

65. `test_cards_remaining_methods`

Why it is needed: Validates that the Cards remaining methods functionality operates securely without boundary violations.

Result: `Passed`

66. `test_cards_empty`

Why it is needed: Validates that the Cards empty functionality operates securely without boundary violations.

Result: `Passed`

67. `test_property_add_not_existing`

Why it is needed: Validates that the Property add not existing functionality operates securely without boundary violations.

Result: `Passed`

68. `test_cards_reshuffle`

Why it is needed: Validates that the Cards reshuffle functionality operates securely without boundary violations.

Result: `Passed`

69. `test_main_cli`

Why it is needed: Validates that the Main cli functionality operates securely without boundary violations.

Result: `Passed`

70. `test_game_property_auction_branch`

Why it is needed: Validates that the Game property auction branch functionality operates securely without boundary violations.

Result: `Passed`

71. **`test_game_bankruptcy_branches`**

Why it is needed: Ensures bankruptcy processing correctly handles nested player elimination paths and properties are released gracefully.

Result: `Passed (Fixed)`

Bug/Error: AttributeError: 'Player' object has no attribute 'in_jail' AND Error 15: Removing a player from the `players` list shifted all subsequent indices, causing the game loop to skip the next player's turn completely.

Fix: Corrected the jail attribute access, and implemented a compensation logic in `_check_bankruptcy` that decrements `self.current_index` when the turn player is eliminated to keep the turn rotation consistent.

72. **`test_game_pay_rent_bankrupt`**

Why it is needed: Ensures bankruptcy processing correctly handles nested player elimination paths and properties are released gracefully.

Result: `Passed`

73. **`test_game_interactive_menus`**

Why it is needed: Validates that the Game interactive menus functionality operates securely without boundary violations.

Result: `Passed`

74. **`test_game_trade_fully`**

Why it is needed: Ensures property exchanges correctly validate ownership and fund transfers.

Result: `Passed`

75. **`test_game_trade_logic`**

Why it is needed: Ensures property exchanges correctly validate ownership and fund transfers.

Result: `Passed (Fixed)`

Bug/Error: AssertionError: assert 1500 == (1500 + 10)

Fix: Added `seller.add_money(cash_amount)` in `trade` method so the seller receives the cash from the buyer

76. **`test_game_move_and_resolve_branches`**

Why it is needed: Validates that the Game move and resolve branches functionality operates securely without boundary violations.

Result: `Passed`

77. **`test_fuzz_game_loops`**

Why it is needed: Validates that the Fuzz game loops functionality operates securely without boundary violations.

Result: `Passed`

78. **`test_pay_rent_unowned`**

Why it is needed: Validates that the Pay rent unowned functionality operates securely without boundary violations.

Result: `Passed`

79. **`test_mortgage_already_mortgaged`**

Why it is needed: Checks mortgage multipliers and nested active boolean state transitions.

Result: `Passed`

80. **`test_unmortgage_not_mortgaged`**

Why it is needed: Checks mortgage multipliers and nested active boolean state transitions.

Result: `Passed`

81. **`test_unmortgage_cannot_afford`**

Why it is needed: Checks mortgage multipliers and nested active boolean state transitions.

Result: `Passed (Fixed)`

Bug/Error: TypeError: unsupported operand type(s) for *: 'method' and 'float' AND Error 16: Properties were being unmortgaged even if the player could not afford the cost, as the state was changed before the balance check.

Fix: Fixed the type conversion for mortgage value calculation, and added a state revert `prop.is_mortgaged = True` if the player fails the final affordability check during unmortgaging.

82. **`test_find_winner_empty_players`**

Why it is needed: Validates that the Find winner empty players functionality operates securely without boundary violations.

Result: `Passed`

83. `test_interactive_menu_invalid`

Why it is needed: Validates that the Interactive menu invalid functionality operates securely without boundary violations.

Result: `Passed`

84. `test_menu_trade_player_selection`

Why it is needed: Ensures property exchanges correctly validate ownership and fund transfers.

Result: `Passed`

85. `test_main_coverage`

Why it is needed: Validates that the Main coverage functionality operates securely without boundary violations.

Result: `Passed`

86. `test_game_run_empty`

Why it is needed: Validates that the Game run empty functionality operates securely without boundary violations.

Result: `Passed`

87. `test_game_move_railroad`

Why it is needed: Validates that the Game move railroad functionality operates securely without boundary violations.

Result: `Passed`

88. `test_game_interactive_all_menu_branches`

Why it is needed: Validates that the Game interactive all menu branches functionality operates securely without boundary violations.

Result: `Passed`

89. `test_game_menu_trade_empty_props`

Why it is needed: Ensures property exchanges correctly validate ownership and fund transfers.

Result: `Passed`

90. `test_card_deck_repr_non_empty`

Why it is needed: Verifies the non-empty `CardDeck.__repr__` branch returns a valid index-based summary string.

Result: `Passed`

91. **`test_game_bankruptcy_index_reset_when_no_players_left`**

Why it is needed: Ensures `_check_bankruptcy` correctly resets `current_index` when the final player is eliminated.

Result: `Passed`

92. **`test_game_bankruptcy_index_clamped_when_too_large`**

Why it is needed: Ensures `_check_bankruptcy` clamps an out-of-range positive `current_index` back to `0` after elimination.

Result: `Passed`

93. **`test_game_bankruptcy_index_clamped_when_below_minus_one`**

Why it is needed: Ensures `_check_bankruptcy` clamps an invalid very negative `current_index` to `-1`.

Result: `Passed`